KubeSQL

我们来做一个 Go 写的小工具，暂时叫 **ksql**。

Intro

Kubernetes 资源本来就是结构化数据。比如一个 Deployment，有名字、命名空间、副本数、标签和状态。我们完全可以给这些字段起几个“列名”，把一组 Deployment 看成一张表。

SQL 里的东西	在这个项目里的意思
表	一种 Kubernetes 资源，比如 deployments
一行	一个资源对象，比如 default 命名空间里的 web
一列	资源的某个字段，比如 name、namespace、replicas
SELECT	从 API Server 读取对象，再投影成需要的列
WHERE	筛选符合条件的对象
INSERT	请求 API Server 创建对象
UPDATE	请求 API Server 修改对象
DELETE	请求 API Server 删除对象

在这个项目中，我们无需考虑关系型数据库的ACID约束原则和一些其他特性，具体来说：

- 你**无需**保证一个操作是原子的。直接调用 Apiserver 就可以，剩下的交给他
- 你**无需**保证一致性，但是最好不要一个语句就把你的集群弄炸了）
- 你**无需**考虑事务、并发控制
- 你**无需**考虑多表联合查询，不过如果以下任务你都做完了，也可以尝试一下

Requirements

我们对你实现项目的方式有一些要求：

1. **实现方式。** 全程用 Go。SQL 的词法分析器逐字符扫描，语法分析器用递归下降；不能用正则表达式、按空格切分、关键词切片。Kubernetes 访问用官方 client-go，程序本身不通过执行 kubectl 来干活。
2. **连接方式。** 读取 kubeconfig，支持 --kubeconfig、--context、--namespace。没传 namespace 时，用当前 context 的命名空间；context 没配置时用 default。SELECT 可以通过 --all-namespaces 查询全部命名空间；本任务书的写操作只在一个明确的命名空间里执行。Namespace 这类集群级资源不受 --namespace 影响。
3. **输出方式。** 提供 --output json，查询结果始终是 JSON 数组。数字输出为数字，NULL 输出为 JSON null。除非另有说明，测试按行集合比较，不要求行顺序，也不要求 JSON 对象的键顺序。
4. **测试怎么跑。 你需要自己准备一个正常运行的 Kubernetes 集群（推荐使用 minikube）。** 每章给出的“准备 YAML”同时服务于该章的测试。把它保存为文件，用下面的方式准备环境：

   代码块

   ```
   kubectl --context <测试集群> create -f fixture.yaml
   ksql --context <测试集群> --namespace <用例里的命名空间> --output json < query.sql
   ```

5. **怎么提交。**请创建一个 Github 仓库提交你的任务。保持 gofmt 格式，运行 go test ./...、go vet ./... 和 staticcheck ./...；用 go fix、与项目 Go 版本兼容的 modernize、deadcode 检查可现代化和可删除的代码，确认改动后再提交。实现放在少量职责清楚的包里，CLI 可以先用标准库 flag。

   开发时按功能少量多次提交，通常一个 commit 不超过 200 行，使用不带 emoji 的 Angular 风格消息，例如 feat(parser): parse select statements、test(query): cover null filtering。不用为了凑提交数拆开一个不可分割的小改动。
6. **怎么使用 AI 辅助编程。**我们鼓励你使用 Coding Agent 来进行开发，不过请你务必熟悉你的 Agent 写出来的所有东西的概念（什么是语法树？什么是CRD？）

Tasks

easy：小小的递归下降解析器

要想使用 SQL 语句写查询，我们得先弄懂 SQL 是怎么变成可以执行的查询的。在这个过程中，我们会用到一种叫 递归下降 的分析方法（请自行搜索这是什么），将一条 SQL 语句转换为对应的语法树（这

是什么？也搜索一下吧）。

我们之所以把这个任务设置为 easy，是希望你能善用 AI 工具理解复杂概念，并编写代码。当然，如果你很熟悉对应的概念也很享受手写代码，直接上手也不是不可以（

在这一步中，你可以只实现以下语法：

代码块

```
statement  = selectStmt, [";"], EOF ;
selectStmt = "SELECT", ("*" | columnList), "FROM", identifier ;
columnList = identifier, {",", identifier} ;
```

EBNF 里的大写单词代表关键字，匹配时忽略大小写。SELECT * 用一个明确的星号节点表示，不要在解析阶段擅自展开字段。

这一任务只做语法分析。表名是否存在、列是否合法，留给后面的语义检查。但是，你需要考虑后续增加不同 SQL 语法的复杂度。

测试 1-1：正常生成 AST

输入：

代码块

```
select name, replicas FROM deployments;
```

预期 AST：

代码块

```
{
  "type": "select",
  "columns": [
    {"type": "column", "name": "name"},
    {"type": "column", "name": "replicas"}
  ],
```

```
  "table": "deployments"
}
```

测试 1-2：输出错误位置

输入：

代码块

```
SELECT name, FROM deployments;
```

预期错误：

代码块

```
{
  "code": "E_PARSE",
  "line": 1,
  "column": 14,
  "message": "这里需要列名，却遇到了 FROM"
}
```

easy：连接 K8s

现在，让我们正式进入 K8s 的世界！你需要在这一步接入集群 API ，查询真实的数据。

接上 Kubernetes API，先只支持这三张表：

表名	API 资源	easy 阶段的列
namespaces	core/v1 的 Namespace	name
deployments	apps/v1 的 Deployment	name、namespace、replicas

ingresses	networking.k8s.io/v1 的 Ingress	name、namespace、default_backend_service

其中：

- name、namespace 对应 metadata.name、metadata.namespace。
- replicas 对应 spec.replicas。
- default_backend_service 对应 spec.defaultBackend.service.name；没有默认后端，或默认后端不是 Service 时，这一列是 NULL。
- SELECT * 返回本阶段表里列出的全部列。
- 表名只支持表格里的复数写法；别急着兼容 deploy、ns 之类缩写。
- 未知表、未知列在请求集群前报错。查不到对象就返回 []。

两个用例共用的准备 YAML

代码块

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-easy-select
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: sql-easy-select
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
---
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: sql-easy-select
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  namespace: sql-easy-select
spec:
  defaultBackend:
    service:
      name: web
      port:
        number: 80
```

测试 2-1：查询集群级资源

输入：

代码块

```sql
SELECT name FROM namespaces;
```

预期输出中至少包含这一行：

代码块

```
[{"name":"sql-easy-select"}]
```

集群通常还会有 default、kube-system 等 Namespace，因此这里检查“包含该行”，不检查“只有该行”。传 --namespace sql-easy-select 不会过滤 Namespace 表。

测试 2-2：查询当前命名空间里的 Deployment 和 Ingress

两个输入分别执行，CLI 使用 --namespace sql-easy-select。

输入 A：

```
代码块
1   SELECT name, namespace, replicas FROM deployments;
```

预期输出 A：

```
代码块
1   [{"name":"web","namespace":"sql-easy-select","replicas":2}]
```

输入 B：

```
代码块
1   SELECT * FROM ingresses;
```

预期输出 B：

```
代码块
1   [{"name":"web","namespace":"sql-easy-select","default_backend_service":"web"}]
```

easy：过滤！

光支持查询怎么够？我们还需要进行一些筛选！

在 SELECT 后面加 WHERE。范围先控制在：

- 比较：=、<>、>、>=、<、<=。
- 逻辑：AND、OR、NOT。
- 括号：用来改变结合顺序。
- 空值：IS NULL、IS NOT NULL。
- 值：字符串、整数、小数、TRUE、FALSE、NULL，以及列引用。

比较和 IS NULL 先结合，然后是 NOT、AND、OR。比如：

```
代码块
1   name = 'web' OR name = 'worker' AND replicas >= 3
```

必须解释为：

```
代码块
1   (name = 'web') OR ((name = 'worker') AND (replicas >= 3))
```

求值也别偷懒成“全转字符串再比较”：10 要大于 2，数字和不兼容的字符串比较要报类型错误，不能按字典序蒙过去。

NULL 按 SQL 的三值逻辑处理：比较可能得到 TRUE、FALSE、UNKNOWN，WHERE 只保留 TRUE。包括 NOT UNKNOWN 仍然是 UNKNOWN。字段没设置是 NULL；列名写错则是错误，两者别混在一起。

你可以使用两种方式来实现过滤：

- 推荐：Kubernetes fieldSelector，将 WHERE 限定的查询直接转换为 K8s API Server 的过滤条件

- 简单：直接拉取所有资源，然后在客户端根据规则做过滤

实现任意一种即可

两个用例共用的准备 YAML

代码块

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-easy-where
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: sql-easy-where
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: worker
  namespace: sql-easy-where
spec:
  replicas: 3
  selector:
    matchLabels:
      app: worker
  template:
    metadata:
      labels:
        app: worker
```

```
    spec:
      containers:
        - name: worker
          image: nginx:1.27
---
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: sql-easy-where
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: internal
  namespace: sql-easy-where
spec:
  rules:
    - host: internal.example.test
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

测试 3-1：AND、OR 和括号不能算错

CLI 使用 --namespace sql-easy-where。

输入 A：

```sql
SELECT name FROM deployments
WHERE name = 'web' OR name = 'worker' AND replicas >= 3;
```

预期输出 A：

代码块

```
[{"name":"web"},{"name":"worker"}]
```

输入 B：

代码块

```sql
SELECT name FROM deployments
WHERE (name = 'web' OR name = 'worker') AND replicas >= 3;
```

预期输出 B：

代码块

```
[{"name":"worker"}]
```

测试 3-2：NULL 不能用等号判断

CLI 使用 --namespace sql-easy-where。internal 的规则后端虽然是 web，但它没有默认后端；不要把 rules 里的 Service 名称填进 default_backend_service 列。

输入 A：

代码块

```sql
SELECT name FROM ingresses WHERE default_backend_service IS NULL;
```

预期输出 A：

```
代码块
1   [{"name":"internal"}]
```

输入 B：

```
代码块
1   SELECT name FROM ingresses WHERE default_backend_service = NULL;
```

预期输出 B：

```
代码块
1   []
```

medium：支持 UPDATE 和 DELETE

现在，我们开始支持写入！在这个阶段，我们仍然只操作 Namespace、Deployment 和 Ingress。

我们需要支持以下形式的语句：

```
代码块
1   UPDATE deployments SET replicas = 3 WHERE name = 'web';
2   DELETE FROM deployments WHERE name = 'web';
```

UPDATE 支持多个赋值，用逗号分开。WHERE 直接复用第 4 章的表达式解析和求值，不再写第二套过滤逻辑。

为了方便大家编写，我们只需要支持修改以下资源：

表	可以 SET 的列
namespaces	labels、annotations
deployments	replicas、labels、annotations
ingresses	default_backend_service、labels、annotations

labels、annotations 是 JSON 对象列，输入用包含 JSON 的 SQL 字符串，例如 annotations = '{"owner":"alice"}'。对已知 JSON 列做类型转换，不增加 SQL 函数。SELECT 也可以读取这两列，输出 JSON 对象；各表的 SELECT * 随已公开的列一起扩展。labels 和 annotations 整列赋值表示替换该映射；没有被 SET 点名的字段必须保留。

name、namespace、status 和服务器管理的元数据不能通过本章的 UPDATE 修改。replicas 需要是非负整数。default_backend_service 只接受合法的非空 Service 名称，并且本章只修改已有的 defaultBackend.service.name，保留原端口；没有 Service 类型默认后端时返回明确错误，不自动补一份后端配置，也不允许把必填的 Service 名称设为 NULL。

请注意以下事项：

- 解析、列检查和赋值类型检查在写入前完成；UPDATE、DELETE 缺少 WHERE 时返回 E_WHERE_REQUIRED。
- 从一个命名空间查询候选对象，再用 WHERE 筛选。匹配 0 个对象是成功，affected_rows 为 0。
- 修改只落到指定字段。可以构造 JSON Patch，先 test 读到的 resourceVersion，再对目标字段 add、replace 或 remove；不要拿一份简化结构覆盖整个对象。给 labels、annotations 整列赋值时要替换整个映射，不能因误用 Merge Patch 而把旧键保留下来。
- 多对象操作逐个执行并记录结果。失败对象要带资源类型、namespace、name 和原因；失败后仍处理其余已匹配对象，最后返回汇总，退出码非 0。
- 全部成功时输出 {"affected_rows":N}。有失败时输出 {"affected_rows":S,"failed_rows":F,"errors":[...]}。
- DELETE 使用普通删除，不自动移除 finalizer。只要 API 接受了我们的请求就视为成功，不需要等待完全删除以后再返回

两个用例共用的准备 YAML

```
代码块
apiVersion: v1
kind: Namespace
metadata:
  name: sql-medium-write
  annotations:
    owner: before
---
apiVersion: v1
kind: Namespace
metadata:
  name: sql-medium-delete
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: sql-medium-write
  annotations:
    keep: unchanged
spec:
  replicas: 0
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
---
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: sql-medium-write
spec:
  ports:
    - port: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-v2
  namespace: sql-medium-write
```

```
spec:
  ports:
    - port: 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: temporary
  namespace: sql-medium-write
spec:
  defaultBackend:
    service:
      name: web
      port:
        number: 80
```

测试 4-1：修改三个资源，其他字段别动

CLI 使用 --namespace sql-medium-write。以下每条语句独立执行。

输入：

代码块

```
UPDATE deployments SET replicas = 2 WHERE name = 'web';
```

代码块

```
UPDATE namespaces SET annotations = '{"owner":"alice"}'
WHERE name = 'sql-medium-write';
```

代码块

```
UPDATE ingresses SET default_backend_service = 'web-v2'
WHERE name = 'temporary';
```

三次预期输出分别都是：

代码块

```
{"affected_rows":1}
```

再分别查询确认：

代码块

```
SELECT name, replicas, annotations FROM deployments WHERE name = 'web';
```

代码块

```
[{"name":"web","replicas":2,"annotations":{"keep":"unchanged"}}]
```

代码块

```
SELECT name, annotations FROM namespaces WHERE name = 'sql-medium-write';
```

代码块

```
[{"name":"sql-medium-write","annotations":{"owner":"alice"}}]
```

代码块

```
SELECT name, default_backend_service FROM ingresses WHERE name = 'temporary';
```

代码块

```
[{"name":"temporary","default_backend_service":"web-v2"}]
```

另外检查 Deployment 的镜像、selector、Pod template 仍和修改前一致；Ingress 的默认后端只有 Service 名称变成 web-v2，端口仍是 80，其他未赋值字段保持不变。replicas 变成 2 就满足这里的要

求，不需要等待 Pod Ready。

测试 4-2：删除三个资源

从准备状态开始，CLI 使用 --namespace sql-medium-write。以下语句分别执行：

代码块

```
DELETE FROM ingresses WHERE name = 'temporary';
```

代码块

```
DELETE FROM deployments WHERE name = 'web';
```

代码块

```
DELETE FROM namespaces WHERE name = 'sql-medium-delete';
```

三次预期输出分别都是：

代码块

```
{"affected_rows":1}
```

然后用 kubectl wait --for=delete 等待这三个指定对象消失，测试超时设为 120 秒。等待完成后分别执行：

代码块

```
SELECT name FROM ingresses WHERE name = 'temporary';
```

代码块

```
SELECT name FROM deployments WHERE name = 'web';
```

代码块

```
SELECT name FROM namespaces WHERE name = 'sql-medium-delete';
```

三次预期输出分别都是：

代码块

```
[]
```

medium：创建！

接下来我们需要创建资源。

创建 Deployment 时，光有 name 和 replicas 不够，还要有 selector、Pod template、容器镜像。为了不在这一章设计一大堆“快捷列”，我们给三张表各增加一个 JSON 类型的列：**manifest**，表示完整的资源文档。

采用标准的 INSERT 结构：

代码块

```
INSERT INTO 表名 (manifest) VALUES ('一段完整的 JSON 资源文档');
```

SQL 字符串写法保持标准；执行时因为目标列是 JSON 类型，把字符串解析成 JSON 对象。这是本工具对列的类型约定，不是发明一种新的 SQL 运算符。manifest 加入表的可读列，SELECT manifest 返回 JSON 对象，SELECT * 也会包含它。UPDATE manifest 暂时不允许整份覆盖资源，沿用上一章的可写列。

注意两层解析各管各的：SQL lexer 负责单引号和两个单引号的转义，JSON 内容交给 Go 的 encoding/json。不能拿一个正则同时拆 SQL 和 JSON。

这一章先支持一个 VALUES 元组，一次创建一个对象；批量 INSERT 不在本章范围。

创建前检查：

- 这一章是三类持久化资源的创建规则；hard 阶段再按具体 API 能力扩展。
- manifest 必须是合法 JSON 对象，有匹配目标表的 apiVersion、kind 和 metadata.name；暂不支持 generateName。
- Deployment 和 Ingress 没写 metadata.namespace 时用 CLI 选定的 namespace；明确写了且与 CLI 不同则报错，不悄悄跨命名空间创建。
- Namespace 是集群级资源，manifest 不得带非空 metadata.namespace。
- 拒绝用户写入 status、uid、resourceVersion、managedFields 等服务器管理字段。资源本身的必填项和约束交给 API Server 校验，并把原因带回来。
- 同名对象已存在时报 AlreadyExists，不能偷偷改成更新，也不能自动删除重建。

本章测试要特别注意

下面的“目标 YAML”定义了 SQL 应该创建出来的对象，**不要 apply 目标 YAML**。开始用例前，目标对象应不存在。创建后，比较 YAML 中明确写出的字段；API Server 补上的默认值、时间戳和 resourceVersion 不要求与 YAML 完全相同。

测试 5-1：用 INSERT 创建 Namespace 和 Deployment

本例无需预建 Namespace。目标 YAML：

代码块

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-medium-insert
---
apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: api
  namespace: sql-medium-insert
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: nginx:1.27
```

先输入：

代码块

```
INSERT INTO namespaces (manifest)
VALUES ('{"apiVersion":"v1","kind":"Namespace","metadata":{"name":"sql-medium-insert"}}');
```

预期输出：

代码块

```
{"affected_rows":1}
```

等 sql-medium-insert 进入 Active，再以 --namespace sql-medium-insert 输入：

代码块

```
INSERT INTO deployments (manifest)
VALUES ('{
  "apiVersion":"apps/v1",
  "kind":"Deployment",
```

```
    "metadata":{"name":"api","namespace":"sql-medium-insert"},
    "spec":{
      "replicas":2,
      "selector":{"matchLabels":{"app":"api"}},
      "template":{
        "metadata":{"labels":{"app":"api"}},
        "spec":{"containers":[{"name":"api","image":"nginx:1.27"}]}
      }
    }
  }');
```

预期输出：

代码块

```
{"affected_rows":1}
```

查询确认：

代码块

```
SELECT name, namespace, replicas FROM deployments WHERE name = 'api';
```

代码块

```
[{"name":"api","namespace":"sql-medium-insert","replicas":2}]
```

创建出来的 Deployment，其 selector、template 和镜像也应与目标 YAML 一致。

测试 5-2：创建 Ingress，同名再建要失败

准备 YAML，只 apply 这部分：

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-medium-insert-ingress
---
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: sql-medium-insert-ingress
spec:
  ports:
    - port: 80
      targetPort: 80
```

目标 YAML，**不要 apply**：

代码块

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  namespace: sql-medium-insert-ingress
spec:
  ingressClassName: nginx
  defaultBackend:
    service:
      name: web
      port:
        number: 80
```

CLI 使用 --namespace sql-medium-insert-ingress，输入：

代码块

```
INSERT INTO ingresses (manifest)
VALUES ('{
  "apiVersion":"networking.k8s.io/v1",
  "kind":"Ingress",
  "metadata":{"name":"web","namespace":"sql-medium-insert-ingress"},
```

```
    "spec":{
      "ingressClassName":"nginx",
      "defaultBackend":{"service":{"name":"web","port":{"number":80}}}
    }
  }');
```

第一次预期输出：

代码块

```
{"affected_rows":1}
```

立即重复执行同一条 SQL，预期失败，退出码非 0：

代码块

```
{
  "affected_rows": 0,
  "failed_rows": 1,
  "errors": [
    {
      "resource": "ingresses",
      "namespace": "sql-medium-insert-ingress",
      "name": "web",
      "reason": "AlreadyExists"
    }
  ]
}
```

错误信息可以增加 API Server 提供的说明，但 reason 和对象标识应保持可检查。失败后仍然只有一个 web，原有对象没有被覆盖。

hard：支持内置资源

众所周知 K8s 里存在着大量资源，我们也不可能为每一种资源都写上一套完整的处理逻辑。所以接下来，我们需要实现通用的资源查询

在查询一种未知资源时，我们需要先用 API Discovery 获取资源的 group、version、resource、scope 和 verbs，用 client-go 的 dynamic client 与 unstructured 对象执行操作。

这一步覆盖范围包含集群实际提供的内置资源：Pod、Node、Service、ReplicaSet、StatefulSet、DaemonSet、Job、CronJob、ConfigMap、Secret、PV、PVC、StorageClass、ServiceAccount、Role、ClusterRole、各类 Binding 等。

表名的约定：

- 无歧义时可以写 configmaps、statefulsets。
- 精确指定资源时，使用一个双引号标识符，例如 "apps/v1/statefulsets"、"v1/configmaps"。
- 核心组用 v1/资源名；其他组用 group/version/资源名。
- 只写复数资源名时，先按组判断歧义，再选择该组中实际提供该资源的 preferred version；preferred version 没有该资源时，按 Discovery 的版本优先顺序找第一个提供它的版本。

再做通用字段访问。 所有对象至少能通过 name、namespace、labels、annotations、manifest 访问。集群级对象的 namespace 是 NULL。

更深的字段使用双引号包起来的 JSON Pointer 作为列名，例如：

代码块

```
SELECT name, "/spec/replicas" AS replicas FROM statefulsets;
UPDATE configmaps SET "/data/mode" = 'prod' WHERE name = 'settings';
```

这里 SELECT、AS、UPDATE、SET 以及双引号标识符的语法都是普通 SQL 写法。

读取不存在的路径得到 NULL。有效路径读到对象或数组时，输出对应的 JSON；本项目的 WHERE 比较仍只支持标量；写入路径缺失时只允许在已存在的父对象下添加叶子，不自动创建一串父对象。

这一章增加标准形式的 CAST('...' AS JSON)，用来明确赋值对象、数组、布尔值或 JSON null，例如：

代码块

```
UPDATE configmaps
SET "/data" = CAST('{"mode":"prod","region":"test"}' AS JSON)
```

```
3   WHERE name = 'settings';
```

这会替换 data 整个对象。普通 SQL 字符串仍然是字符串，不要自动猜它是不是 JSON。CAST 只需支持本任务要求的字符串转 JSON，不顺手扩成一个通用函数系统。

SQL NULL 用在 UPDATE 的可选字段上表示移除字段；CAST('null' AS JSON) 则表示保存一个 JSON null，最终是否允许由资源 schema 决定。数字处理要保留整数精度，不能把所有资源数值先转成 float64。

通用写操作沿用 medium 的 WHERE 和失败汇总规则。INSERT 继续通过 manifest 提交完整资源。UPDATE 禁止改 apiVersion、kind、name、namespace、status 和服务器管理字段；也要拦住覆盖这些字段的祖先路径，例如不能整块替换 /metadata 来绕过限制。每种资源的其他不可变字段交给 API Server 拒绝，不自动删了重建。Secret 的 data 保持 API 中的 Base64 表示，错误日志不要输出完整资源或 Secret 数据。

测试 6-1：读取常见资源

准备 YAML：

代码块

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-hard-native
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: settings
  namespace: sql-hard-native
data:
  mode: dev
---
apiVersion: v1
kind: Secret
metadata:
  name: credentials
  namespace: sql-hard-native
```

```
type: Opaque
stringData:
  username: demo
  password: not-a-real-secret
---
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web-rs
  namespace: sql-hard-native
spec:
  replicas: 0
  selector:
    matchLabels:
      app: web-rs
  template:
    metadata:
      labels:
        app: web-rs
    spec:
      containers:
        - name: web
          image: nginx:1.27
---
apiVersion: v1
kind: Service
metadata:
  name: cache
  namespace: sql-hard-native
spec:
  clusterIP: None
  selector:
    app: cache
  ports:
    - port: 80
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: cache
  namespace: sql-hard-native
spec:
  serviceName: cache
  replicas: 0
  selector:
    matchLabels:
      app: cache
```

```yaml
  template:
    metadata:
      labels:
        app: cache
    spec:
      containers:
        - name: cache
          image: nginx:1.27
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data
  namespace: sql-hard-native
spec:
  storageClassName: ""
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

PVC 不需要绑定成功，ReplicaSet 和 StatefulSet 的副本数为 0，因此本例不依赖存储供应器和 Pod 调度。

CLI 使用 --namespace sql-hard-native。分别输入下列 SQL，右边是对应的完整预期结果：

输入 SQL	预期输出
SELECT name FROM replicasets;	[{"name":"web-rs"}]
SELECT name, "/spec/replicas" AS replicas FROM "apps/v1/statefulsets";	[{"name":"cache","replicas":0}]
SELECT name, "/data/username" AS username FROM secrets;	[{"name":"credentials","username":"ZGVtbw=="}]

SELECT name, "/spec/resources/requests/storage" AS storage FROM persistentvolumeclaims;	[{"name":"data","storage":"1Gi"}]
SELECT name, "/data/mode" AS mode FROM configmaps;	[{"name":"settings","mode":"dev"}]

再输入：

代码块

```
UPDATE configmaps SET "/data/mode" = 'prod' WHERE name = 'settings';
```

代码块

```
{"affected_rows":1}
```

代码块

```
SELECT name, "/data/mode" AS mode FROM configmaps WHERE name = 'settings';
```

代码块

```
[{"name":"settings","mode":"prod"}]
```

hard：支持 kubectl top

kubectl top 读取的是 Metrics API 的实际观测值，Pod 的 requests、limits 不是实际用量。这里增加两张只读表：

表	列	含义

pod_metrics	name、namespace、cpu_millicores、memory_bytes	每个 Pod 一行，累加它所有容器的指标
node_metrics	name、cpu_millicores、memory_bytes	每个 Node 一行，直接使用 NodeMetrics

实现 kubectl top pod 和 kubectl top node 的 CPU、内存查询能力即可，不要求复制它的全部命令行选项。

CPU 先累加 Quantity，再转换为整数 millicores；内存转换为整数 bytes，直接用 Kubernetes 的 resource.Quantity。

Metrics API 通常由 Metrics Server 提供，需要相应服务和读取权限。未安装或不可用时返回 E_METRICS_UNAVAILABLE；403 返回权限错误；指标还没采到时明确报告无样本或返回对应空结果，不能伪造为 0。

测试 7-1：读取 top 数据

准备 YAML：

代码块

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-hard-metrics
---
apiVersion: v1
kind: Pod
metadata:
  name: measure
  namespace: sql-hard-metrics
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      resources:
        requests:
```

```
            cpu: 10m
            memory: 16Mi
          limits:
            cpu: 200m
            memory: 128Mi
      - name: sidecar
        image: busybox:1.36
        command: ["sh", "-c", "sleep 3600"]
        resources:
          requests:
            cpu: 5m
            memory: 8Mi
          limits:
            cpu: 100m
            memory: 128Mi
```

CLI 使用 --namespace sql-hard-metrics，分别输入：

代码块

```
SELECT name, cpu_millicores, memory_bytes FROM pod_metrics
WHERE name = 'measure';
```

代码块

```
[{"name":"measure","cpu_millicores":150,"memory_bytes":83886080}]
```

代码块

```
SELECT name, cpu_millicores, memory_bytes FROM node_metrics
WHERE name = 'test-node';
```

代码块

```
[{"name":"test-node","cpu_millicores":250,"memory_bytes":536870912}]
```

创建上面的 Pod，确认 Pod Ready、Metrics API 可用，再等指标出现，最长等待 180 秒。再次执行 Pod 查询，预期恰好有一行 name 为 measure，CPU、内存为非负整数。具体值可以和样例输出不同。

hard：支持 CRD

上一章的我们实现了k8s自带资源的读取。这一章，我们要做到：集群出现新资源后，工具无需增加专用 Go 类型，也不用重新编译。

先分清两个东西：

- CustomResourceDefinition 是“定义一种资源”的对象。
- Custom Resource 是“按这个定义创建出来的实例”。

我们需要保证两者都要能操作。其他通过聚合 API 提供的资源，也走 Discovery 和 dynamic client。不能靠资源名称猜复数，不能假设所有资源都在某个固定 API 组，更不能假设它们都有 spec.replicas。

具体补齐这些能力：

- 按 Discovery 返回的实际资源名、scope 和 verbs 路由操作，字段通过 manifest 和 JSON Pointer 访问。
- 多个 served version 按规则选择；显式指定版本时只访问该版本。
- 如果做了 Discovery 缓存，遇到未知资源或过期映射时刷新一次再解析。没有缓存也可以，不为这个要求专门做常驻服务。
- 创建 CRD 和创建 CR 分成两步，中间等待就绪；不能把一次性 apply 多文档当成可靠的注册等待机制。
- API 的 schema 校验、准入拒绝、RBAC Forbidden、字段不可变和版本冲突都要向上返回，不能统一改成“SQL 语法错误”。
- 某个 API 组发现失败时，已知的其他组仍可使用，但失败组不能当作“本来就不存在”；错误要说明发现请求失败。

这里的“所有资源”指集群 Discovery 实际提供的资源类型。每种资源是否支持 SELECT、INSERT、UPDATE、DELETE，要看它的 verbs 和服务器行为：只读资源、只支持 create 的 Review 类 API，都不能硬套完整 CRUD。SELECT 至少需要 list；本方案的条件写入需要 list 和对应的 patch 或 delete；

INSERT 需要 create。不具备所需 verb 时返回 E_UNSUPPORTED_VERB。对 Review 这类请求对象，INSERT 按其 API schema 提交，不再强加 medium 阶段持久化资源必须有 metadata.name 的规则。

带 /status、/scale 的子资源，以及 exec、logs、portforward 等流式或连接接口，不是这个任务里的普通资源表。识别它们并说明不支持即可；用户仍能读到主资源 manifest 里返回的 status。

测试 8-1：一种完全陌生的资源，照样能增删改查

先创建这个 Namespace 和 CRD：

代码块

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: sql-hard-crd
---
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: gadgets.lab.example.com
spec:
  group: lab.example.com
  scope: Namespaced
  names:
    plural: gadgets
    singular: widget
    kind: Widget
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["size"]
              properties:
                size:
                  type: integer
                  minimum: 1
```

```
32                   message:
33                     type: string
```

等待：

代码块

```
kubectl  wait --for=condition=Established crd/gadgets.lab.example.com --timeout=60s
```

确认 Discovery 已能看到 gadgets，再创建实例 YAML：

代码块

```
apiVersion: lab.example.com/v1
kind: Widget
metadata:
  name: sample
  namespace: sql-hard-crd
spec:
  size: 2
  message: hello
```

CLI 使用 --namespace sql-hard-crd。

先查询：

代码块

```
SELECT name, "/spec/size" AS size
FROM "lab.example.com/v1/gadgets"
WHERE "/spec/size" >= 2;
```

代码块

```
[{"name":"sample","size":2}]
```

再修改：

代码块

```
UPDATE "lab.example.com/v1/gadgets"
SET "/spec/size" = 3 WHERE name = 'sample';
```

代码块

```
{"affected_rows":1}
```

再次查询 sample，预期 size 为 3，message 仍然是 hello。然后用 INSERT 创建第二个实例；它的目标 YAML 如下，不要提前创建：

代码块

```
apiVersion: lab.example.com/v1
kind: Widget
metadata:
  name: extra
  namespace: sql-hard-crd
spec:
  size: 5
```

输入：

代码块

```
INSERT INTO "lab.example.com/v1/gadgets" (manifest)
VALUES ('{
  "apiVersion":"lab.example.com/v1",
  "kind":"Widget",
  "metadata":{"name":"extra","namespace":"sql-hard-crd"},
  "spec":{"size":5}
}');
```

代码块

```
{"affected_rows":1}
```

最后删除它：

代码块

```
DELETE FROM "lab.example.com/v1/gadgets" WHERE name = 'extra';
```

代码块

```
{"affected_rows":1}
```

等 extra 消失后查询这张表，预期只剩：

代码块

```
SELECT name, "/spec/size" AS size FROM "lab.example.com/v1/gadgets";
```

代码块

```
[{"name":"sample","size":3}]
```

测试 8-2：新发现的集群级资源，不能被 namespace 绊住

先创建 CRD：

代码块

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: clusternotes.lab.example.com
spec:
  group: lab.example.com
```

```
  scope: Cluster
  names:
    plural: clusternotes
    singular: clusternote
    kind: ClusterNote
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                owner:
                  type: string
```

等待它 Established，并确认 Discovery 可见，再创建实例：

代码块

```
apiVersion: lab.example.com/v1
kind: ClusterNote
metadata:
  name: cluster-note
spec:
  owner: platform
```

CLI 故意传 --namespace sql-hard-crd，输入：

代码块

```
SELECT name, namespace, "/spec/owner" AS owner
FROM "lab.example.com/v1/clusternotes"
WHERE name = 'cluster-note';
```

预期输出：

代码块

```
[{"name":"cluster-note","namespace":null,"owner":"platform"}]
```

再输入：

代码块

```
SELECT name FROM "apiextensions.k8s.io/v1/customresourcedefinitions"
WHERE name = 'clusternotes.lab.example.com';
```

预期：

代码块

```
[{"name":"clusternotes.lab.example.com"}]
```